Ames Housing Study

Authors: Haneen Khaled, Mustafa Elbahrawy, Omar Mohammed – **October 2026**

1. Motivation

Accurate house pricing and validation is vital for buyers, sellers & accessors. This project aims for the following:

- Conducting explorartory data analysis (EDA), and exploring how different machine learning algorithms handle inter-feature relations within real estate data
- Building a series of robust preprocessing and training pipelines to predict residential property prices in Ames, Iowa, US

2. Dataset Overview and Preprocessing

Dataset shape: 1460 rows × 79 columns (excluding `Id` & `SalePrice` target)

Feature types:

- 36 Numerical Features: Discrete count data and continuous linear/area measurements (e.g., `GrLivArea`, `TotalBsmtSF`, `LotFrontage`, `GarageCars`).
- 16 Ordinal Categorical Features: Quality and condition ratings with an inherent natural order (e.g., `ExterQual`, `BsmtQual`, `HeatingQC`, `KitchenQual`).
- 27 Nominal Categorical Features: Qualitative variables without order (e.g., `Neighborhood`, `Foundation`, `MSZoning`, `SaleType`).

2.1. Train-test Split

Starting with the following train-test split, configured with the following ratios for training, testing & cross-validation respectively **(70:15:15)**

```
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.15, random_state=SEED
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=round(0.15 / 0.85, 6), random_state=SEED
)
```

2.2. Skewness Handling

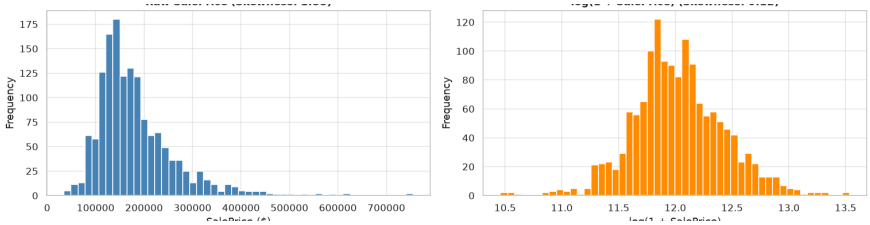


Figure 2.1: Price target transformation via `np.log1p` Log transformer

,

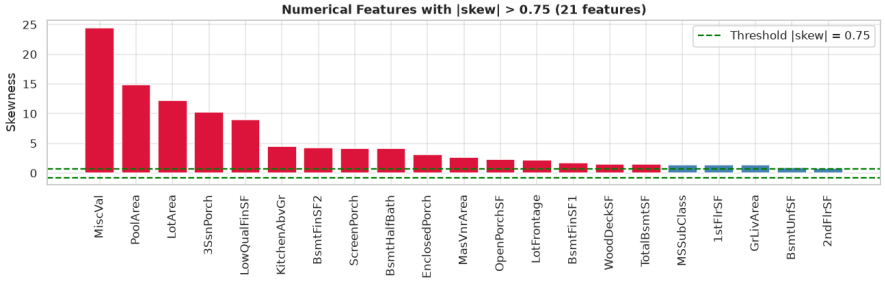
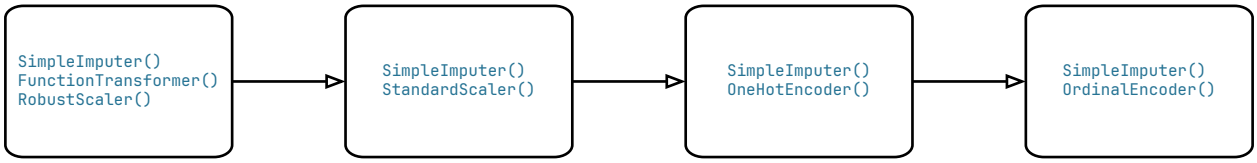


Figure 2.2: Exploring skewness for multiple numerical features, where ones with `skewness factor > 0.75` are marked in red rather than blue

2.3. Preprocessing Pipeline



A **4-phase preprocessing pipeline**, with phases numbered in order from left to right

Phase 1: Transforming skewed numerical features
Phase 2: Handling non-skewed features
Phase 3: Nominal Encoding
Phase 4: Ordinal Encoding

3. Exploratory Data Analysis and Insights

In this section, we're going to discuss and answer some questions based on insights from our dataset after loading it via `pd.read_csv('ames.train.csv')`. Now make sure to follow up with the insights by running the embedded code blocks here

3.1. Temporal & Seasonality Analysis

```
    corrDf = df[['MoSold', 'SalePrice']].corr()
    print(corrDf, "\n") -> (1)

    print(df['MoSold'].value_counts().sort_index())

    monthly_averages = df.groupby('MoSold')['SalePrice'].mean().astype(int).sort_values(ascending=False)
    print(f"${monthly_averages}") -> (2)
```

(1): Returns all months sorted by sales count per month. **June** & **July** have the highest sales count overall

(2): Returns all months sorted by sales average per month. Notice how **September** has the highest sales average. Meanwhile, on the other hand, **April** witnesses the lowest sales average recorded among all months

3.2. Neighborhood Analysis

(1): Comparing house prices across neighborhoods

```
    neigh_avg = df.groupby('Neighborhood')
['SalePrice'].mean().sort_values(ascending=False)

    print("\nTop 3 Neighborhoods:")
    for n, p in neigh_avg.head(3).items():
        print(f"  {n:<12}: ${p:,.0f}")

    print("\nBottom 3 Neighborhoods:")
    for n, p in neigh_avg.tail(3).items():
        print(f"  {n:<12}: ${p:,.0f}")
```

```
    Top 3 Neighborhoods:
      NoRidge     : $335,295
      NridgHt     : $316,271
      StoneBr     : $310,499

    Bottom 3 Neighborhoods:
      BrDale      : $104,494
      IDOTRR      : $100,124
      MeadowV     : $98,576
```

3.3. House Characteristics

(1): How basements significantly affect house prices ?!

```
    has_bsmt = (df['TotalBsmtSF'] > 0).map({True: 'Has
Basement', False: 'No Basement'})
    bsmt_avg = df.groupby(has_bsmt)['SalePrice'].mean()
    diff = bsmt_avg['Has Basement'] - bsmt_avg['No
Basement']

    print(f"\nAverage Price with Basement    :
${bsmt_avg['Has Basement']:,.0f}")
    print(f"Average Price without Basement :
${bsmt_avg['No Basement']:,.0f}")
    print(f"Conclusion: Having a basement adds an
average premium of ${diff:,.0f}.")
```

```
    Average Price with Basement    : $182,878
    Average Price without Basement : $105,653
    Conclusion: Having a basement adds an
    average premium of $77,225.
```

4. Model Evaluation and Benchmarks

The models selected for our training workflow were as follows:

- Linear Regression
- Support Vector Regression (Optimized with `GridSearchCV`)
- Gradient Boosting

4.1. Hyperparameter Tuning and Metrics

SVR:

$$C = [0.01 - 10], \gamma = [0.1 - 10] \text{ or auto,} \\ \text{kernels} = [\text{RBF, Linear}], \text{ degrees} = [2, 3] \tag{4.1}$$

```
svr = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', SVR(C=0.76, epsilon=0.09))
])

svrParamGrid = {
    'regressor__C': [0.01 ,0.1, 1, 10],
    'regressor__kernel': ['rbf', 'linear'],
    'regressor__gamma': ['auto', 0.1, 1, 10],
    'regressor__degree': [2,3],
}

svrGrid = GridSearchCV(
    estimator=svr,
    param_grid=svrParamGrid,
    cv=5,
    scoring='r2',
    n_jobs=-1
)
svrGrid.fit(X_train, y_train)
```

Gradient Boosting:

$$\text{Loss: Huber, Learning rate} = 0.04 \\ \text{max depth} = 3, \text{n_estimators} = 174 \tag{4.2}$$

```
    gbr = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('regressor', GradientBoostingRegressor(
        loss='huber', learning_rate=0.04,
n_estimators=174,
        max_depth=3))
    ])

    gbr.fit(X_train, y_train)
```

Linear Regression

```
    Lr = Pipeline(steps=[
    ('preprocessor',
        preprocessor),
    ('regressor',
        LinearRegression())
    ])

    Lr.fit(X_train, y_train)
```

4.1.1. Comparison Table

Model	Train R^2	Test R^2	MAE	MSE	RMSE
Linear Regression	0.949042	0.900177	0.085252	0.016979	0.130302
SVR	0.932661	0.920870	0.076755	0.013459	0.116012
Gradient Boosting	0.947598	0.900920	0.083474	0.016852	0.129816

> **Note**
>
> Aforementioned numbers in the table above are approximations with probable error margins, consider applying regularization or hyperparameter tuning to improve generalization on the test set.

4.2. Final Model Selection and Conclusion

Based on the comparative evaluation across training and test splits, Support Vector Regression (SVR) optimized via `GridSearchCV` is selected as the final model due to its superior generalization capabilities ($R^2 = 0.9209$), lowest prediction error (RMSE = 0.1160), and minimal overfitting variance compared to Linear Regression and Gradient Boosting.